

第 11 章 进程、作业、信号与调度优先级

从“看到一个 PID”到可审计的运行控制：先确认实例，再观察状态，最后实施最小干预。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航 先抓住一条主线：**进程控制不是先找一条“能杀掉它”的命令，而是先证明目标进程是谁、当前处于什么状态、由谁管理，再选择最小影响的操作并用同一组证据复核。**

01 确认运行实例PID、用户、启动时间、完整命令行

02 还原关系PPID、线程、PGID、SID、TTY

03 判断状态R、S、D、T、Z 与资源维度

04 持续采样ps 快照、top 趋势、/proc 下钻

05 实施最小干预作业控制、TERM、nice / renice

06 分层验收原 PID、原业务特征、非目标实例、管理者

专题地图 | 类型 | 专题 | |---|---| | 知识专题 | 从程序文件到运行实例：进程、线程与身份关系 | | 知识专题 | 进程状态、生命周期与资源维度 | | 操作专题 | 用 `ps`、`pidof` 与 `pstree` 建立可信快照 | | 操作专题 | 用 `pgrep` 精确缩小集合，再决定是否操作 | | 操作专题 | 用 `top` 和 `/proc/` 建立持续证据 | | 操作专题 | Bash 前台、后台与作业生命周期 | | 操作专题 | 信号控制：先请求协作，再决定是否强制 | | 操作专题 | `nice` 与 `renice` 调整普通调度倾向 | | 诊断专题 | 从状态和负载症状推进到下一条证据 | | 经典任务 | 安全停止旧 worker；调查高负载无单一热点 |

阅读时持续回答 1. 当前 PID 仍然属于刚才确认的业务实例吗？ 2. 这个对象是单个进程、线程、进程组，还是 Shell 作业？ 3. `STAT` 是瞬时状态，还是持续性异常证据？ 4. 负载来自可运行竞争，还是不可中断等待？ 5. 当前命令会影响一个 PID，还是一个匹配集合？ 6. 命令成功能证明请求已发送，还是终态已达到？ 7. 原 PID 消失后，原业务特征是否出现了新 PID？ 8. 当前对象是否受 systemd 或其他监督器管理？

阅读边界：本章建立进程实例和 Shell 作业的调查与控制模型；systemd unit 生命周期留给第 12 章，日志证据留给第 13 章，cgroup 深度治理不在 RHCSA 主线展开。

第 11 章 · 正文

服务器上出现“程序正在运行”“负载很高”“进程杀不掉”“SSH 断开后任务也没了”时，真正需要处理的不是某一条孤立命令，而是一组容易混淆的运行对象：程序文件、进程实例、线程、父子关系、进程组、会话、控制终端和 Shell 作业。把命令名当作唯一身份、把 load average 当作 CPU 百分比、把 kill 返回成功当作进程已经退出，都会让一次看似简单的处理变成误伤或反复试错。

本章以“身份组合 → 状态与资源 → 查询证据 → 最小操作 → 再验证 → 管理层交接”为主线。先用 PID、PPID、用户、启动时间和完整命令行建立可复核的实例身份，再区分 R/S/D/T/Z、RSS、VSZ、I/O 和负载；随后训练 ps、pgrep、top、/proc、Shell 作业控制、信号以及 nice 值。若进程被外部管理者重新拉起，本章只负责识别这一边界，完整 systemd 控制留到下一章。

遇到任何“停掉它”或“调低它”的请求，先写出四个问题：- 我要操作的是哪个实例，身份由哪些字段共同证明？- 作用范围是单个 PID、进程组、作业，还是匹配集合？- 当前证据支持终止、暂停、继续，还是只支持继续调查？- 操作后怎样证明目标达到终态、非目标未受影响、管理者没有重新创建它？

概念

进程实例（process instance）是程序一次实际运行形成的内核对象，拥有 PID、凭据、地址空间、打开文件和当前状态。同一个可执行文件可以同时形成多个实例，因此程序路径或短命令名只能说明“它可能是什么”，不能单独证明“它就是本次要操作的对象”。

概念

线程（thread）是进程内部参与调度的执行单元；同一进程的线程通常共享地址空间和多数资源。进程级视图适合定位业务实例，线程级视图适合继续定位同一实例内部的 CPU 热点；线程转储和应用栈分析则超出本章边界。

概念

PID 与 PPID 分别标识当前进程实例和它的直接父进程。PID 只在进程存活期间唯一，退出后可能被复用；因此高风险操作前必须重新核对用户、启动时间、完整命令行和 PPID，不能把几分钟前记录的裸 PID 当作长期业务身份。

概念

进程组（process group）用 PGID 把若干进程组织成一个控制对象。一条 Shell 管道通常包含多个 PID，却位于同一进程组；终端的前台属性和由键盘触发的信号通常面向前台进程组，而不是只面向管道中的某个成员。

概念

会话与控制终端（`session / TTY`）连接了登录环境、进程组和终端控制。SID 标识会话，TTY 表示控制终端；`TTY=?` 常见于守护进程、systemd 服务或已经脱离终端的任务，它只说明没有控制终端，并不自动表示异常。

概念

进程状态（`process state`）是采样瞬间任务处于运行、可中断等待、不可中断等待、停止或已退出待回收等状态的压缩表示。状态字符不是健康评分：`S` 往往正常，`D` 需要调查等待依赖，`Z` 表示父进程尚未回收，均不能用一条更强的 `kill` 统一处理。

概念

Shell 作业（`job`）是当前交互 Shell 对一条命令或管道的管理记录。`%1` 这样的 jobspec 只在创建它的 Shell 中有效；它不是 PID，也不是跨会话的持久身份。后台运行、脱离终端和长期服务是三个不同问题。

概念

信号（`signal`）是内核向进程或进程组传递的异步通知。`SIGTERM` 是可协作处理的终止请求，`SIGKILL` 是无清理机会的强制终止；`SIGSTOP` / `SIGCONT` 控制停止与继续。发送成功只证明请求被内核接受，不证明业务终态已经达到。

概念

nice 值（`niceness`）表示普通 CPU 调度中的相对礼让程度，常见范围为 `-20` 到 `19`：数值越低越有利，越高越礼让。它不是 CPU 配额，也不能修复 I/O 等待、锁竞争、内存压力或错误的管理策略。

操作语义以下入口分别负责“建立身份、缩小集合、保存趋势、读取上下文、控制作业、发送信号和调整调度倾向”。先理解作用对象，再记关键形式。

`ps` / `pidof` / `pstree`

SYNOPSIS

```
ps [selection] -o FIELDS [--sort=KEY]
pidof [options] PROGRAM
pstree [options] [PID|USER]
```

读取进程快照、快速取得程序 PID，或观察父子结构。`ps` 的核心是把“选谁”和“显示什么”分开。

``-e` / `-p PID``

选择全部进程，或只选择明确 PID。

``--ppid PID``

选择直接子进程，不递归整个后代树。

``-o pid,ppid,user,lstart,stat,args``

建立可复核的身份、关系和状态视图。

``--sort=-%cpu` / `-sort=-rss``

按 CPU 或 RSS 降序排列；负号表示降序。

``pgrep` / `pskill``

SYNOPSIS

```
pgrep [options] PATTERN
pskill [options] PATTERN
```

按名称、用户、父进程、会话、终端或完整命令行缩小集合；`pskill` 对同一集合发送信号。

``-a``

显示 PID 和完整命令行，适合在操作前人工审阅。

``-f` / `-x``

匹配完整命令行，或要求短命令名精确匹配。

``-u USER` / `-P PPID``

按有效用户或直接父 PID 收缩范围。

同条件预览

任何 `pskill` 前先运行对应 `pgrep -a`，之后再用原条件验收。

``top``

SYNOPSIS

```
top [-b] [-d SECONDS] [-n ITERATIONS] [-H] [-p PID]
```

周期采样系统和任务状态；批处理模式可把多个采样窗口保存为证据。

``-b``

批处理输出，适合重定向到文件。

``-d 2 -n 5``

每 2 秒采样一次，共 5 轮；先固定窗口再比较。

``-H``

展开线程视图，继续定位进程内部热点。

``-p PID``

只观察指定进程；与 `-H` 组合可观察该进程的线程。

`/proc/<PID>`

SYNOPSIS

```
cat /proc/PID/status
tr '\0' ' ' < /proc/PID/cmdline
readlink /proc/PID/exe
ls -l /proc/PID/fd
```

读取活进程的身份、状态、命令行、路径、文件描述符、I/O 和等待点。

``status` / `stat``

查看人类可读状态，或紧凑的机器字段。

``cmdline` / `exe` / `cwd``

确认参数、真实可执行文件和工作目录。

``fd` / `io` / `wchan``

观察打开资源、I/O 计数和当前等待位置。

活对象竞态

进程可在读取途中退出；“文件不存在”不等于它从未存在。

``jobs` / `bg` / `fg``

SYNOPSIS

```
jobs [-l]
bg [JOBSPEC]
fg [JOBSPEC]
```

管理当前交互 Shell 的作业表，把停止的作业放到后台继续，或拉回前台。

``jobs -l``

同时显示作业号和 PID，连接 Shell 与内核视图。

``bg %N``

向指定作业发送继续信号并在后台运行。

``fg %N``

把指定作业放回终端前台并等待。

``%N``

jobspec 只属于当前 Shell，不可带到另一个登录会话使用。

``nohup` / `disown``

SYNOPSIS

```
nohup COMMAND [ARG]... &
disown [-h] [JOBSPEC]
```

分别处理挂断信号和 Shell 作业表关系；二者都不等于把命令配置成长期系统服务。

``nohup COMMAND &``

让命令忽略常见挂断影响，并明确放入后台。

``disown %N``

从当前 Shell 作业表移除作业。

``disown -h %N``

保留作业记录，但标记为不随 Shell 发送 HUP。

边界

重启持久性、依赖、自动恢复和统一日志应交给 systemd。

``kill``

SYNOPSIS

```
kill [-SIGNAL] PID|JOBSPEC ...  
kill -0 PID  
kill -SIGNAL -- -PGID
```

向明确 PID、jobspec 或进程组发送信号；默认信号通常是 ``SIGTERM``。

``-TERM`` / ``-KILL``

先请求有序退出；重新确认后才考虑无清理机会的强制终止。

``-STOP`` / ``-CONT``

无条件停止，或继续已停止的任务。

``-0``

检查 PID 存在和发送权限，不验证业务身份或健康。

``-- -PGID``

负目标表示进程组，可能同时影响整条管道或一个作业。

``nice`` / ``renice``

SYNOPSIS

```
nice -n ADJUSTMENT COMMAND [ARG]...  
renice --priority VALUE --pid PID  
renice --priority VALUE --pgrp PGID  
renice --priority VALUE --user USER
```

在启动时或运行中调整普通 CPU 调度倾向；只有证据支持 CPU 竞争时才是合适的最小干预。

``nice -n N``

在继承值基础上增加调整量，常见默认调整量为 10。

``--priority VALUE``

为运行中对象设置绝对 nice 值，避免 ``-n`` 兼容语义歧义。

``--pid`` / ``--pgrp`` / ``--user``

明确后续标识符是 PID、进程组还是用户集合。

权限

普通用户通常只能让自己的任务更礼让；提高调度有利程度需要适当特权。

[知识专题] 从程序文件到运行实例：进程、线程与身份关系

进程调查最容易犯的错误，是把命令名当成唯一身份。真实系统里，同名 worker、预派生服务、解释器进程和多线程应用都很常见。可靠的切入点不是“找到一个看起来像的 PID”，而是建立可复核的身份组合，并理解这个实例与父进程、线程、进程组和会话的关系。

① [知识点] 程序、进程与线程是三个层次

程序文件是可执行内容；进程是该内容的一次运行实例，拥有独立的地址空间、凭据、打开文件、环境和状态；线程是进程中的调度执行单元，同一进程的线程通常共享地址空间和多数资源。

进程级视图适合回答“哪个业务实例占资源、由谁启动、是否仍存在”；线程级视图适合继续回答“同一个进程内部，哪个执行单元消耗 CPU”。`ps -eL`、`ps -T -p PID` 或 `top -H -p PID` 可以展开线程，但应用栈、Java 线程转储或数据库内部执行计划不属于本章。

② [知识点] PID 是当前实例标识，不是长期业务身份

每个进程都有 PID 和 PPID。PID 在进程存活期间唯一，但退出后可被复用。单独把 PID 写入笔记、脚本或延迟执行的命令，都可能形成检查—操作竞争窗口。

对高风险操作，至少组合以下字段：

字段	回答的问题
PID	当前实例是谁
PPID	直接由谁创建
USER / UID	以谁的身份运行
LSTART / ETIMES	何时启动、运行多久
COMM	内核记录的短命令名
ARGS / CMD	包含参数的完整命令行
SID / PGID / TTY	属于哪个会话、进程组和终端

因此，终止前的可靠核对不是 `ps -p 2450` 一眼扫过，而是明确读取需要证明身份的字段：

```
ps -p 2450 -o pid,ppid,user,lstart,etimes,sid,pgid,TTY,stat,args
```


③ [知识点] 父子关系解释创建、退出与回收

进程通常由父进程创建。子进程退出后，父进程应读取其退出状态并回收剩余的进程表项。若子进程已经退出而父进程尚未回收，就会暂时显示为僵尸。

```
父进程创建子进程
→ 子进程运行
→ 子进程退出并保留退出状态
→ 父进程 wait 回收
→ 进程表项消失
```

如果父进程先退出，仍存活的子进程会被 PID 1 或某个子收割器接管。所谓“孤儿”只描述父进程关系改变，不等于进程失控，也不等于必须终止。

④ [知识点] 一条管道可包含多个进程，但只有一个 Shell 作业

Bash 执行一条管道时，通常会启动多个进程，并把它们放入同一进程组。Shell 把整个管道登记为一个作业。终端维护一个前台进程组；用户按下 `Ctrl+C` 或 `Ctrl+Z` 时，信号通常发送给整个前台进程组，而不是只发送给管道最左侧的一个 PID。

这解释了三个常见现象：

- `jobs` 只显示一个作业，但 `ps` 可看到多个 PID；
- 前台管道中的多个命令会同时收到终端信号；
- 对单个 PID 发送信号，可能只影响管道中的一个成员。

⑤ [知识点] SID、PGID 与 TTY 连接了会话和终端控制

- `SID` 是会话 ID；登录 Shell 通常是会话首进程。
- `PGID` 是进程组 ID；一条管道的成员通常位于同一进程组。
- `TTY` 是控制终端；`?` 常表示进程没有控制终端。
- 终端的“前台”属性属于进程组，不属于某个孤立 PID。

守护进程、由 `systemd` 启动的服务和某些脱离终端的后台任务常显示 `TTY=?`。这本身不是异常，只说明它们不受当前终端的前后台控制。

[Cheatsheet] 终止或调优前同时核对 PID、用户、启动时间、完整命令行和 PPID；一条管道可有多个 PID，但 Bash 可把它视为一个 Job；终端信号通常面向前台进程组。

[知识专题] 进程状态、生命周期与资源维度

状态字符不是健康评分，而是采样瞬间“任务当前处于哪一类运行或等待状态”。资源字段也不能相互替代：CPU、物理内存、虚拟地址空间、I/O 和负载平均值分别回答不同问题。先把这些维度拆开，才能避免“看到一个大数字就直接杀进程”。

① [知识点] R、S、D、T、Z 的核心含义

状态	含义	第一判断
R	正在 CPU 上运行或位于可运行队列	持续大量出现才支持 CPU 竞争假设
S	可中断睡眠，等待事件	服务大部分时间处于 S 往往正常
D	不可中断睡眠，常见于内核 I/O 等待	先查等待点、存储、NFS、驱动和内核证据
T	被作业控制信号停止，或处于跟踪状态	查终端、作业、调试器与信号来源
Z	子进程已退出，父进程尚未回收状态	查 PPID 和父进程，不要继续“杀僵尸”

STAT 后还可能出现修饰符：**s** 表示会话首进程，**l** 表示多线程，**+** 表示位于前台进程组，**<** 表示较高调度优先级，**N** 表示较低调度优先级。

② [知识点] 僵尸已经退出，剩下的是父进程的回收责任

僵尸已释放普通地址空间和大部分资源，只保留 PID、退出状态等少量进程表信息。对僵尸发送 **SIGKILL** 没有意义，因为可执行实体已经退出。

正确调查链是：

```
找到 Z 状态 PID
→ 读取 PPID
→ 检查父进程是否仍存在、是否健康
→ 判断父进程是否持续产生未回收子进程
→ 再决定修复父进程或其管理单元
```

单个短暂僵尸可能在下一次采样中消失；持续累积才说明父进程没有正常回收。

③ [知识点] D 状态不是“信号更强就会立即消失”

D 表示任务处于不可中断等待。信号可被标记为待处理，但通常要等当前内核等待返回后才产生效果。因此，不断升级到 **SIGKILL** 不能替代 I/O 调查。

可优先观察：

```
ps -eo pid,ppid,stat,wchan:28,etimes,args | awk '$3 ~ /^D/'
cat /proc/<PID>/wchan
cat /proc/<PID>/io
findmnt
```

wchan 只是当前等待位置的线索，不应单独当作根因结论。系统日志属于第 13 章；本章只说明它是进一步核对内核、块设备或网络文件系统异常的入口。

④ [知识点] RSS 与 VSZ 不能直接互换

- **RSS** 是当前驻留在物理内存中的页面量。
- **VSZ** 是进程虚拟地址空间总量，可能包含共享库、映射文件、尚未实际驻留的区域和保留地址空间。
- **%MEM** 通常以 RSS 相对物理内存的比例计算。

因此，“VSZ 很大”不能直接推出“实际占用了同等 RAM”。内存异常需要结合 RSS 趋势、共享映射、系统可用内存和交换活动继续判断。

⑤ [知识点] load average 不是 CPU 使用百分比

uptime 和 **top** 通常显示最近 1、5、15 分钟的负载平均值。Linux 负载同时考虑处于可运行状态的任务和某些不可中断等待任务。因此负载高可能来自：

- 多个 **R** 状态任务竞争 CPU；
- 多个 **D** 状态任务等待本地或远程 I/O；
- 短时间并发峰值；
- 线程或短命子进程数量异常。

没有单个高 CPU 进程，不代表系统不存在 CPU 竞争；同样，CPU idle 仍有余量，也不代表高负载一定是“误报”。必须结合任务状态和多轮采样区分假设。

⑥ [知识点] 单次快照只是一帧，趋势才支持持续性判断

一次 **ps** 输出只能证明采样瞬间。短命进程、高频线程和波动型 CPU 热点可能在下一次采样中完全不同。需要保存证据时，应固定采样间隔和轮数，而不是不断手工刷新后凭印象判断。

```
top -b -d 2 -n 5 -w 200 > /tmp/top-sample.txt
```

[Cheatsheet] **R** 看可运行竞争，**D** 看内核等待，**T** 看暂停来源，**Z** 查父进程；RSS 是驻留物理内存，VSZ 是虚拟地址空间；load average 不是 CPU 百分比。

[操作专题] 用 **ps**、**pidof** 与 **pstree** 建立可信快照

ps 的价值不在于背诵某一套默认输出，而在于把“选择哪些进程”和“显示哪些字段”分开。调查前应先明确问题：是找同名实例、查看父子树、确认启动时间，还是按 CPU、内存和状态排序。

① [操作] 使用 **-e**、**-p**、**--ppid**、**-u** 与 **-c** 选择对象

作用对象：内核当前维护的进程快照。

基本形式：**ps** [选择选项] **-o** <FIELDS> [**--sort**=<KEY>]

关键边界：**-C** 按短命令名选择，不等同于匹配完整命令行；**--ppid** 只选直接子进程。

```
ps -e -o pid,ppid,user,stat,ni,%cpu,%mem,rss,etimes,args
ps -p 2450 -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,args
ps --ppid 2450 -o pid,ppid,stat,etimes,args
ps -u appsvc -o pid,ppid,stat,lstart,etimes,args
ps -C report-worker -o pid,ppid,user,stat,etimes,args
```

② [操作] 用 `-o` 和 `--sort` 让输出服务于当前问题

CPU 调查、内存调查和身份核对需要不同字段：

```
# CPU 与运行状态
ps -eo pid,ppid,user,stat,ni,psr,%cpu,etimes,args --sort=-%cpu

# 内存
ps -eo pid,ppid,user,%mem,rss,vsz,stat,etimes,args --sort=-rss

# 会话和终端
ps -eo pid,ppid,sid,pgid,tpgid,tty,stat,args --sort=sid,pgid,pid
```

字段太多会降低可读性。现场可使用 `ps L` 或 `ps --help output` 查看可用字段，再按问题组合。

③ [操作] 用 `ps --forest` 与 `pstree` 观察父子结构

```
ps -e --forest -o pid,ppid,user,stat,etimes,args
pstree -ap
pstree -aps 2450
```

`pstree -a` 显示参数，`-p` 显示 PID，`-s` 显示目标进程的祖先路径。相同子树可能被压缩显示，因此需要逐个 PID 的精确字段时仍回到 `ps`。

父子树可以揭示：

- worker 是否由同一个主进程派生；
- 目标是否由 Shell、计划任务或服务管理器启动；
- 杀掉一个子进程后主进程是否可能重建；
- 某个高负载进程是否只是更大进程树的一部分。

④ [操作] `pidof` 适合快速入口，不承担完整身份确认

```
pidof chronyd
pidof -s chronyd
```

`pidof` 适合查询已知程序名对应的 PID 列表；`-s` 只返回一个 PID。但它不会自动证明“这个 PID 就是业务题目所指实例”。多实例、解释器脚本和参数差异仍需用 `ps` 或 `pgrep -a -f` 继续核对。

⑤ [验证点] 操作后用同一字段集合重新取证

若操作前使用下面的身份视图：

```
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
```

操作后也应复用同样的字段和原匹配条件。不要只看“原 PID 是否消失”，还要检查原业务特征是否出现新 PID。

[Cheatsheet] 选择范围与输出字段分开；`-o` 定制字段，`--sort` 排序；`pstree` 看结构，`pidof` 只是快速入口；变更前后使用同一身份字段和匹配条件。

[操作专题] 用 `pgrep` 精确缩小集合，再决定是否操作

`pgrep` 适合把系统中的全部进程收缩为一个可审阅集合。真正的安全边界不是“会不会写正则”，而是先预览将被操作的对象，并理解不同条件之间如何组合。`pkill` 与 `pgrep` 使用相近的选择语义，因此每一条批量信号命令都应有对应的预览命令。

① [操作] 短命令名、完整命令行与精确匹配

```
pgrep -l report-worker      # PID + 短命令名
pgrep -a report-worker      # PID + 完整命令行显示
pgrep -a -x report-worker   # 短命令名精确匹配
pgrep -a -f '/opt/report/bin/worker --queue=night'
```

`-f` 把匹配范围扩展到完整命令行，适合区分同一程序的参数实例，也更容易误匹配。模式应尽量包含稳定路径和关键参数，进入 `pkill` 前必须先用完全相同的条件预览。

② [操作] 按用户、父进程、进程组、会话和终端收缩范围

```
pgrep -a -u appsvc report-worker
pgrep -a -P 2450
pgrep -a -g 6200
pgrep -a -s 6000
pgrep -a -t pts/2
```

常用选择条件：

选项	选择对象
<code>-u</code>	有效用户 ID
<code>-U</code>	真实用户 ID
<code>-P</code>	直接父 PID

选项	选择对象
<code>-g</code>	进程组 ID
<code>-s</code>	会话 ID
<code>-t</code>	控制终端
<code>-r</code>	运行状态集合

同一选项中的逗号列表通常表达多个候选值；不同选择条件同时存在时，目标必须满足全部条件。不要把条件叠加误解为“任意一个命中即可”。

③ [操作] 使用最旧、最新、运行时长和线程选择器

```
pgrep -a -o report-worker      # 最旧实例
pgrep -a -n report-worker      # 最新实例
pgrep -a -O 3600 report-worker # 启动超过 3600 秒
pgrep -w report-worker         # 列出线程 ID
```

“最旧”或“最新”只是排序条件，不自动等于“旧业务实例”或“正确目标”。仍应核对用户、父进程、完整参数和启动时间。

④ [操作] 把预览结果交给 `ps` 进行第二层确认

```
mapfile -t pids < <(pgrep -u appsvc -f '/opt/report/bin/worker --queue=old')
((${#pids[@]})) || { echo 'no candidates'; exit 0; }

ps -p "${IFS=,; echo "${pids[*]}"}" \
  -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,args
```

这里的目标不是要求考试中必须写复杂 Shell，而是形成思维顺序：匹配集合只是候选，身份字段才是操作依据。

⑤ [安全边界] `pkill` 前后都使用原条件验证

```
pgrep -a -u appsvc -f '/opt/report/bin/worker --queue=old'
pkill -TERM -u appsvc -f '/opt/report/bin/worker --queue=old'
sleep 2
pgrep -a -u appsvc -f '/opt/report/bin/worker --queue=old'
```

若最终查询出现新 PID，不应直接再次 `pkill`。应比较启动时间和 PPID，判断是否由主进程、systemd、容器或其他监督器重新创建。

[Cheatsheet] `-x` 精确短命令名，`-f` 完整命令行，`-u/-P/-g/-s/-t` 按属性收缩；不同条件通常是 AND；`pkill` 前必须用同条件 `pgrep -a` 预览，之后再原条件验收。

[操作专题] 用 top 和 /proc/<PID> 建立持续、可下钻的证据

ps 是一次快照，top 是周期采样，/proc 是进程级上下文。三者并不是互相替代，而是从系统概览逐步收缩到具体对象：先观察负载和任务状态分布，再选择 PID 或线程，最后读取命令行、工作目录、打开文件和 I/O 等上下文。

① [操作] 用批处理模式保存多轮 top 证据

```
top -b -d 2 -n 5 -w 200 > /tmp/top-5x2s.txt
```

参数	作用
-b	非交互批处理模式
-d 2	两轮间隔 2 秒
-n 5	共输出 5 轮后退出
-w 200	扩大输出宽度，减少命令行截断
-o %CPU	指定排序字段

固定间隔和轮数可以让前后对比使用相同采样窗口。首次采样中的百分比可能受累计统计影响，不应只取第一屏下结论。

② [操作] 按 PID、用户或线程下钻

```
top -p 2450
top -H -p 2450
top -b -H -p 2450 -d 2 -n 5 -w 200
top -u appsvc
```

-H 展开线程。进程总 CPU 高时，线程视图可继续定位热点；进程总 CPU 不高但负载高时，线程视图也可能揭示多个中等热点。线程 ID 不是另一个独立业务实例，操作前仍应理解其所属进程。

③ [操作] /proc/<PID>/status 与 stat 提供结构化身份和状态

```
cat /proc/2450/status
cat /proc/2450/stat
```

status 适合人工读取，包含名称、状态、PID、PPID、线程数、UID/GID、内存等字段。stat 是单行机器接口，字段数量多，命令名还可能包含空格和括号；没有可靠解析器时，不建议用简单的空格切割脚本硬解析全部字段。

④ [操作] 命令行、可执行文件与工作目录确认真实上下文

```
tr '\0' ' ' < /proc/2450/cmdline; echo
readlink -f /proc/2450/exe
readlink -f /proc/2450/cwd
```

`cmdline` 使用 NUL 分隔参数，直接 `cat` 可能显示为粘连文本。内核线程或已退出进程可能没有预期内容。`exe` 和 `cwd` 受权限、命名空间和进程生命周期影响，读取失败要区分“对象已消失”和“当前用户无权访问”。

⑤ [操作] 文件描述符与 I/O 帮助解释“它在等什么”

```
ls -l /proc/2450/fd
cat /proc/2450/io
cat /proc/2450/wchan
```

`fd/` 可显示打开的文件、管道和套接字链接；`io` 提供读写计数；`wchan` 显示当前内核等待位置。它们都是线索：文件描述符存在不证明业务读写正常，I/O 计数增长也不证明数据已持久化到远端系统。

⑥ [边界] `/proc` 是活对象接口，读取天然存在竞态

进程可在读取多个文件之间退出或执行新程序，因此：

- 某个文件刚读取成功，下一条路径可能已经不存在；
- 对高风险操作，应在发送信号前再次读取关键身份；
- 不要把 `/proc/PID` 消失等同于业务终态正确；
- 不要默认打印 `environ`，其中可能包含令牌、密码或其他敏感信息。

[Cheatsheet] `top -b -d -n` 保存趋势，`-H` 展开线程；`status` 看人类可读状态，`cmdline` 用 NUL 分隔，`exe/cwd` 看真实路径，`fd/io/wchan` 看资源与等待线索；活对象读取存在竞态。

[操作专题] Bash 前台、后台与作业生命周期

作业控制只管理当前交互 Shell 启动的命令或管道。它解决的是“在同一个终端会话里怎样暂停、继续和切换前后台”，不提供开机启动、自动重启、依赖、统一日志或资源治理。需要长期运行的业务应迁移到下一章的 `systemd` 模型。

① [操作] 使用 `&` 启动后台作业

```
find /srv -type f -printf '%p\n' > /tmp/srv-files.txt 2>&1 &
jobs -l
```


& 让 Shell 不等待命令完成并立即返回提示符。后台不等于脱离终端：进程仍可能继承终端、收到 HUP，或因尝试读取终端输入而停止。

② [操作] 用 `Ctrl+Z`、`bg` 与 `fg` 切换状态

```
# 前台命令运行时按 Ctrl+Z
jobs -l
bg %1
fg %1
```

`Ctrl+Z` 通常向前台进程组发送 `SIGTSTP`；`bg` 发送继续信号并让作业留在后台；`fg` 把作业重新设置为前台。需要交互输入的程序在后台读取终端时可能再次停止。

③ [知识点] `jobspec` 只属于当前 Shell

引用	含义
<code>%1</code>	作业 1
<code>%+</code> 或 <code>%%</code>	当前作业
<code>%-</code>	上一个作业
<code>%name</code>	以名称前缀匹配的作业

另一个终端没有同一作业表，即使它可以通过 `ps` 看到相同 PID。脚本和非交互 Shell 的作业控制行为也不同，因此不要把 `%1` 当成系统级稳定标识。

④ [操作] `jobs -l` 与 `ps` 分别提供 Shell 和内核视角

```
jobs -l
ps -p <PID> -o pid,ppid,sid,pgid,tpgid,TTY,stat,args
```

`jobs` 显示 `Done` 说明当前 Shell 已获知作业结束；作业从列表消失不保证所有双重 `fork` 的后代都结束。需要确认进程树时继续使用 `pstree` 或按 PGID 查询。

⑤ [操作] `nohup` 与 `disown` 处理终端挂断的不同部分

```
nohup /usr/local/bin/import-data \
  > /var/tmp/import-data.log 2>&1 &

jobs -l
disown -h %1
```

- `nohup` 启动命令时让它忽略 `SIGHUP`，并在标准流仍指向终端时调整重定向。
- `disown -h` 让 Bash 在退出时不向该作业发送 HUP，但保留作业表条目。

- `disown %1` 将作业从作业表移除。

`nohup` 不会自动把命令放到后台，因此常与 `&` 和显式重定向组合。两者都不提供失败重启、依赖关系和持久服务定义。

⑥ [工作迁移] 一次性长任务与长期服务使用不同方案

偶发的数据导入、临时编译和维护脚本，可以在充分记录 PID、输出和退出状态的前提下使用 `nohup`。需要反复运行、系统启动后自动存在、失败后恢复或统一审计的程序，应定义为 `systemd service`；这不是“命令写得更长”的问题，而是生命周期管理对象已经改变。

[Cheatsheet] 后台 `&`；列作业 `jobs -l`；暂停 `Ctrl+Z`；后台继续 `bg %N`；拉回前台 `fg %N`；`jobspec` 只属于当前 Shell；`nohup/disown` 不等于 `systemd` 服务。

[操作专题] 信号控制：先请求协作，再决定是否强制

信号是异步通知，不是“删除进程”的同义词。多数信号可被程序捕获、忽略或改变处理方式；`SIGKILL` 和 `SIGSTOP` 不能被捕获、阻塞或忽略。安全控制必须先确认对象和范围，再发送最小影响的信号，最后重新查询。

① [知识点] 常用信号及边界

信号	常见用途	边界
<code>SIGHUP</code>	终端挂断；部分程序约定为重读配置	是否 reload 由程序定义
<code>SIGINT</code>	交互中断，常来自 <code>Ctrl+C</code>	可被程序处理
<code>SIGTERM</code>	请求有序终止	默认首选，可被处理或忽略
<code>SIGKILL</code>	内核强制终止	无清理机会，只作最后手段
<code>SIGSTOP</code>	无条件停止	不能被程序处理
<code>SIGTSTP</code>	终端停止，常来自 <code>Ctrl+Z</code>	可被处理
<code>SIGCONT</code>	继续停止的进程	用于恢复 <code>T</code> 状态
<code>SIGCHLD</code>	子进程状态变化	父进程据此回收子进程

信号编号在不同体系结构上并非全部一致，命令和文档中优先使用名称。`kill -l` 可查看当前系统的信号名称与编号。

② [操作] `kill` 对明确 PID 或 jobspec 发送信号

```
kill -TERM 2450
kill -CONT 2450
kill -STOP 2450
kill -TERM %1
```

未指定信号时，`kill` 默认发送 `SIGTERM`。Bash 的 `kill` 是 builtin，因此可以直接接受 jobspec。命令返回成功只说明内核接受了发送请求，不证明目标已经退出。

③ [操作] `kill -0` 检查存在与权限，但不是身份验证

```
kill -0 2450
```

信号 0 不执行普通信号动作，可用于检查 PID 是否存在以及当前用户是否有发送权限。但它不能证明：

- PID 仍属于原业务实例；
- 进程处于健康状态；
- 进程已经完成清理；
- 业务功能达到目标。

因此强制操作前仍需重新读取用户、启动时间和命令行。

④ [操作] 向进程组发送信号时明确负 PID 语义

外部 `/usr/bin/kill` 和 Shell builtin 在参数解析上可能存在细节差异。对进程组操作，应使用清晰形式并先确认 PGID：

```
ps -p 2450 -o pid,pgid,sid,tt,stat,args
kill -TERM -- -6200
```

负的目标值表示进程组。它可能影响整条管道或同一作业中的多个进程，不能把 PGID 当成“另一个 PID”。考试或脚本中，优先使用信号名称和 `--` 消除选项歧义。

⑤ [操作] `pkill` 按集合发送信号，但必须先预览

```
pgrep -a -u appsvc -f '/opt/app/bin/worker --queue=old'
pkill -TERM -u appsvc -f '/opt/app/bin/worker --queue=old'
```

`pkill` 的风险来自选择集合，而不是信号本身。模式过宽、用户条件遗漏或完整命令行中的公共参数，都可能让非目标进程一起收到信号。

⑥ [流程] 安全终止的标准链路

```
读取现状
→ 缩小候选集合
→ 确认用户、启动时间、完整命令行与父进程
→ 发送 SIGTERM
→ 等待合理时间
→ 查询原 PID 与原业务特征
→ 若仍存在，再次确认身份和状态
→ 最后才考虑 SIGKILL
```

示例骨架：

```
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
kill -TERM "$pid"
sleep 2
ps -p "$pid" -o pid,ppid,user,lstart,etimes,stat,args
pgrep -a -u appsvc -f '/opt/app/bin/worker --queue=old'
```

⑦ [边界] 服务进程被重新拉起时，应切换管理层

若原 PID 消失但相同业务特征立即出现新 PID，应比较 PPID、启动时间和进程树。可能的原因包括：

- 主进程自动重建 worker；
- systemd 的重启策略；
- 容器运行时或其他监督器；
- 计划任务或人工重复启动。

本章到此停止直接反复 `kill`。若确认属于 systemd unit，后续控制入口应转到第 12 章。

[Cheatsheet] 默认先 TERM，等待并再查询；KILL 无清理机会；STOP/CONT 控制暂停恢复；`kill -0` 只查存在与权限；负目标可表示进程组；`pkill` 前用同条件 `pgrep -a`。

[操作专题] `nice` 与 `renice`：调整普通 CPU 调度倾向

`nice` 值不是 CPU 配额，也不保证某个任务在指定时间完成。它只是普通调度中的相对倾向：数值越低，任务越有利；数值越高，任务越愿意让出 CPU。只有在证据支持 CPU 竞争时，调整 `nice` 才可能是合适的最小干预。

① [知识点] `nice` 值方向与常见范围

现代 Linux 上常见范围为 `-20` 到 `19`：

```
-20  更有利
0    常见默认值
19   更不利
```

“提高 nice 值”意味着让进程更礼让，通常会降低其调度有利程度。不要把“数值更大”和“优先级更高”混在一起。

② [操作] nice 在启动新命令时调整 niceness

```
nice -n 10 /usr/local/bin/build-index
```

GNU `nice -n N` 表示在继承的 `niceness` 基础上增加调整值；未指定 `-n` 时默认调整 10。操作后应查看实际 `NI`：

```
ps -C build-index -o pid,user,ni,pri,stat,%cpu,etimes,args
```

③ [操作] renice 修改运行中的 PID、进程组或用户集合

为避免不同实现和兼容模式中 `-n` 相对/绝对语义的歧义，本章优先使用长选项：

```
renice --priority 15 --pid 2450
renice --priority 10 --pgrp 6200
renice --priority 10 --user batchsvc
```

- `--priority` 指定目标绝对 nice 值；
- `--relative` 指定相对调整量；
- `--pid`、`--pgrp`、`--user` 明确解释后续标识符的类型。

④ [边界] 普通用户通常只能让自己的任务更不利

普通用户一般只能操作自己拥有的进程，并只能把 `niceness` 调整到更不利的方向。把 nice 值降低为更有利的值通常需要适当特权。权限错误不能用反复尝试或宽泛 `sudo` 规则绕过；特权授权归第 10 章。

⑤ [验证点] 同时验证对象身份和实际 NI

```
ps -p 2450 -o pid,ppid,user,lstart,ni,pri,stat,%cpu,args
```

若 PID 在 `renice` 前后发生复用，单看命令返回值可能把操作归因到错误实例。验证时仍要保留身份字段。

⑥ [诊断边界] nice 不能修复 I/O、锁、内存或资源上限问题

当高负载来自 `D` 状态、远程文件系统等待、应用锁竞争或内存回收时，调整 nice 可能几乎没有帮助。`cgroup CPUWeight`、配额、实时调度和 I/O 优先级属于更深层资源治理，不在本章完整展开。

[Cheatsheet] nice 值越低越有利；新命令用 `nice`，运行中对象用 `renice`；优先用 `--priority/--relative` 消除歧义；操作后查 `NI`；它不是 CPU 配额，也不修复 I/O 根因。

[诊断专题] 从状态和负载症状推进到下一条证据

诊断不是把所有命令一次执行完，而是在竞争假设之间选择最有区分度的证据。每一步都应回答：当前证据支持什么、排除什么、下一条证据为什么比其他命令更有效。

① [诊断] `z` 状态持续累积

症状：多个 `Z` 状态持续存在

- 当前证据：PID、PPID、启动时间、父进程身份
- 假设：父进程未 `wait`；父进程卡死；父进程持续异常派生
- 下一条证据：按 `PPID` 查看全部子进程和父进程状态
- 最小修复：修复或按管理层重启父进程，而非杀僵尸
- 再验证：`Z` 数量不再增长，旧表项被回收

```
ps -eo pid,ppid,stat,lstart,etimes,args | awk '$3 ~ /^Z/'
ps --ppid <PPID> -o pid,ppid,stat,lstart,etimes,args
ps -p <PPID> -o pid,ppid,user,stat,lstart,etimes,args
```

② [诊断] `D` 状态进程“杀不掉”

症状：TERM/KILL 后 PID 仍在，STAT 为 `D`

- 当前证据：`wchan`、`/proc/PID/io`、共同挂载或设备
- 假设：本地块设备等待、NFS/网络存储等待、驱动/内核问题
- 下一条证据：同类 `D` 进程是否共享等待点或挂载
- 最小修复：恢复依赖或按存储/网络层处理
- 再验证：进程离开 `D`，待处理信号才可能生效

不要为了“证明命令有效”而持续发送更强信号。

③ [诊断] `T` 状态是预期暂停还是异常遗留

```
ps -p <PID> -o pid,ppid,sid,pgid,tpgid,ttty,stat,args
jobs -l
```

若属于当前 Shell 的停止作业，可用 `bg` 或 `fg` 继续；若无作业记录，则调查是否收到 `SIGSTOP`、是否被调试器跟踪、是否由另一个管理员暂停。不要盲目对所有 `T` 状态执行 `SIGCONT`。

④ [诊断] 负载高但无单一高 CPU 进程

优先并行检查四个假设：

假设	最有区分度的证据
多个任务共同竞争 CPU	多轮 <code>top</code> 的 <code>%Cpu</code> 、 <code>R</code> 数量、多个中高 <code>%CPU</code>
大量不可中断等待	<code>D</code> 状态数量、 <code>wchan</code> 、共同 I/O 或挂载
热点在线程	<code>top -H</code> 或 <code>ps -eL</code>
大量短命子进程	多轮采样与 <code>pstree</code> 的父进程派生模式

`renice` 只有在 CPU 竞争得到支持、且业务允许牺牲该任务进度时，才是可考虑的最小干预。

⑤ [诊断] 原 PID 消失但同名进程再次出现

症状: `kill` 后旧 PID 消失，新 PID 很快出现

- 当前证据: 新旧启动时间、PPID、完整命令行、进程树
- 假设: 父进程重建; `systemd` 重启; 容器监督; 计划任务
- 下一条证据: 新 PID 的祖先和管理归属
- 最小修复: 转到实际管理者，而不是继续追杀 PID
- 再验证: 目标业务实例和管理状态同时达到终态

⑥ [安全边界] 诊断中不使用破坏性捷径替代证据

本章不把以下做法作为默认答案：

- 未确认对象就 `kill -9`；
- 未预览集合就宽泛 `pkill -f`；
- 把所有同名进程一次性结束；
- 发现负载高就立即 `renice` 或重启；
- 读取并展示整个 `/proc/PID/envIRON`；
- 把“命令成功”扩大为业务终态正确。

[Cheatsheet] `Z` 查父进程，`D` 查等待依赖，`T` 查作业和信号来源；高负载先区分 `R`、`D`、线程和短命进程；新 PID 出现时转到实际管理者。

[经典任务] 精确定位并安全停止多个同名 worker 中的旧实例

环境与当前状态

系统中存在用户 `appsvc` 启动的多个 `report-worker`：

- 当前实例使用参数 `--generation=current`；
- 一个旧实例使用参数 `--generation=legacy`；
- 所有实例的短命令名相同；
- 题目不提供固定 PID，因为 PID 会随每次实验变化；

- 旧实例可能由父进程重新创建。

目标终态

- 只请求旧实例有序退出；
- 所有 `--generation=current` 实例保持运行；
- 首次不得使用 `SIGKILL`；
- 操作后证明旧参数匹配集合为空；
- 若出现新的 legacy PID，停止继续发送信号并识别管理者。

限制条件

- 不得直接执行宽泛 `kill -f report-worker`；
- 不得使用 `killall report-worker`；
- 不得只凭短命令名或一个旧 PID 操作；
- 不得把“原 PID 消失”当成完整验收。

验收证据

层次	必须证明
候选集合	只匹配 <code>appsvc</code> 的 legacy 参数实例
身份	用户、启动时间、PPID、完整命令行符合目标
操作	首次发送 <code>SIGTERM</code>
目标终态	legacy 匹配集合为空
非目标保护	current 实例仍存在
管理边界	未出现替代 legacy PID；若出现则已定位祖先

[参考解答] 先固定集合，再终止并检查是否被重新拉起

① 调查：用完整参数和用户建立候选集合

```
pgrep -a -u appsvc -f \  
'/usr/local/bin/report-worker .*--generation=legacy'
```

这一步只建立候选集合。若无输出，当前已经没有符合条件的旧实例；若有多个结果，不能随意挑一个 PID，应继续检查每个实例。

② 身份确认：查看启动时间、父进程和完整命令行

```
mapfile -t legacy_pids < <(
  pgrep -u appsvc -f \
    '/usr/local/bin/report-worker .*--generation=legacy'
)

if (($#legacy_pids[@])); then
  csv=$(IFS=,; echo "${legacy_pids[*]}")
  ps -p "$csv" \
    -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,args
fi
```

确认每一行都是 `appsvc`、完整路径和 `legacy` 参数。还应记录 PPID，便于判断后续是否由同一父进程重建。

③ 非目标基线：记录 `current` 实例

```
pgrep -a -u appsvc -f \
  '/usr/local/bin/report-worker .*--generation=current'
```

这条证据用于证明操作没有误伤当前实例。

④ 操作：逐个向已确认 PID 发送 `TERM`

```
for pid in "${legacy_pids[@]"; do
  ps -p "$pid" -o pid,user,lstart,args=
  kill -TERM "$pid"
done
```

在 `kill` 前重新执行一次 `ps`，用于降低 PID 在调查和操作之间退出并复用的风险。

⑤ 等待与分层验证

```
sleep 2

# 原 PID 是否仍存在
for pid in "${legacy_pids[@]"; do
  ps -p "$pid" -o pid,ppid,user,lstart,stat,args
done

# 原业务特征是否仍存在，包括新 PID
pgrep -a -u appsvc -f \
  '/usr/local/bin/report-worker .*--generation=legacy'

# 非目标 current 实例是否仍在
pgrep -a -u appsvc -f \
  '/usr/local/bin/report-worker .*--generation=current'
```

⑥ 结果分支

- legacy 集合为空、current 仍在：达到目标终态。
- 原 PID 仍在但状态为 **D**：停止升级信号，进入 I/O 等待调查。
- 原 PID 仍在且身份未变、程序拒绝 TERM：重新确认影响后才考虑 SIGKILL。
- 出现新的 legacy PID：比较新 PID 的 PPID、祖先和启动时间；转向父进程或 systemd 管理层，不继续追杀 PID。

典型错误包括：只检查原 PID、用 `pkill -f report-worker` 误伤 current、看到多个结果后直接 `kill -9`，以及忽略父进程重建。

[经典任务] 调查负载高但没有单一高 CPU 进程

环境与当前状态

系统在最近 15 分钟持续报告较高 load average。初次打开 `top` 时：

- 没有一个进程长期占据极高 CPU；
- CPU idle 仍有一定余量；
- 不允许先重启、终止或修改 nice；
- 需要用可保存的证据区分 CPU 竞争、I/O 等待、线程热点和短命进程。

目标终态

形成一份能说明“当前最支持哪个假设、下一步进入哪一层”的调查记录，而不是只罗列命令。若证据不足，应明确保留哪些假设，而非强行给出根因。

限制条件

- 不伪造固定输出值；
- 不把 load average 当作 CPU 百分比；
- 不把 `top` 第一屏当作完整趋势；
- 不把 `renice` 作为未知根因的默认修复。

[参考解答] 用固定采样窗口区分 R、D、线程与短命进程

① 建立系统级基线

```
uptime
nproc
```

记录 1、5、15 分钟负载和逻辑 CPU 数量。二者的比较只用于理解压力尺度，不存在适用于所有系统的单一“超过 CPU 数量就必然故障”阈值。

② 保存多轮 top 样本

```
top -b -d 2 -n 5 -w 200 > /tmp/top-5x2s.txt
```

检查每轮中的：

- CPU user/system/iowait/idle 变化；
- R、S、D 等任务数量；
- 是否多个进程共同获得中等 CPU；
- 热点是否在轮次之间快速更换。

③ 使用定制 ps 同时查看状态、等待点和资源

```
ps -eo pid,ppid,user,stat,wchan:28,ni,psr,%cpu,%mem,rss,vsz,etimes,args \
--sort=-%cpu
```

解释分支：

- 多个 R 且 CPU idle 低：CPU 竞争假设增强；
- 多个 D 且共享相似 wchan：I/O 或依赖等待假设增强；
- 无持续热点但 load 高：线程或短命进程假设仍需检查。

④ 展开线程视图

```
ps -eL -o pid,lwp,ppid,psr,stat,ni,%cpu,comm --sort=-%cpu
# 或对具体 PID
top -b -H -p <PID> -d 2 -n 5 -w 200
```

若同一 PID 下多个线程共同消耗 CPU，进程级 %CPU 可能掩盖内部热点分布。这里只定位到线程，不进入应用栈分析。

⑤ 观察父进程和短命子进程模式

```
pstree -ap
for i in 1 2 3 4 5; do
    date '+%F %T'
    ps -eo pid,ppid,stat,%cpu,etimes,args --sort=-%cpu | head -n 30
    sleep 2
done > /tmp/ps-samples.txt
```

若某个父进程持续产生运行时间很短的子进程，单次快照可能看不到稳定的高 CPU 进程，但大量创建和退出仍会提高系统压力。

⑥ 对可疑 PID 读取 `/proc` 上下文

```
cat /proc/<PID>/status
cat /proc/<PID>/io
cat /proc/<PID>/wchan
tr '\0' ' ' < /proc/<PID>/cmdline; echo
```

对 `D` 状态，比较多个进程是否共享等待点和文件系统；对 CPU 竞争，确认任务身份、用户和业务作用，再决定是否需要业务限流或调整 `nice`。

⑦ 结论格式

调查记录至少写明：

症状：15 分钟负载持续偏高
证据：固定 2 秒间隔、5 轮 `top`；定制 `ps`；线程视图
当前最支持：例如 `D` 状态任务共享同一 `NFS` 等待点
仍未排除：例如短命子进程
下一条证据：进入 `NFS`/存储和内核日志层
当前不执行：`kill`、`restart`、`renice`，因为尚未证明它们针对根因
再验证：修复依赖后使用相同采样窗口比较负载和 `D` 数量

如果证据支持 CPU 竞争且允许降低某个批处理任务的吞吐量，才可以在记录基线后使用 `renice --priority`，并用同样的多轮采样验证影响。

[本章收束] 从“看到一个 PID”迁移到可审计的运行控制

本章真正需要保留的不是一长串参数，而是一套可重复的操作顺序：

身份组合
→ 关系与状态
→ 精确选择
→ 固定窗口采样
→ 最小影响操作
→ 同字段再验证
→ 检查原业务特征与管理者

工作方法

1. 先建立基线。记录目标 PID、用户、启动时间、完整命令行、PPID、PGID、SID、TTY 和状态；高负载场景还要固定采样间隔和轮数。
2. 把选择集合与操作分开。`pgrep`、`ps` 先证明候选集合；`pkill`、`kill`、`renice` 后改变状态。

- 3. 优先最小干预。能用 SIGTERM 请求有序退出，就不直接 SIGKILL；根因未知时不先重启或 renice。
- 4. 复用同一证据。操作前后使用同一字段和同一业务匹配条件，才能比较原实例、替代实例和非目标实例。
- 5. 识别管理层。原 PID 消失却出现新 PID，说明“进程实例”可能不是最终控制对象；应比较 PPID、祖先和启动时间，转向实际管理者。

主要判断表

看到的证据	能支持的判断	不能直接推出	下一条高区分度证据
ps 中 PID 存在	当前采样时存在一个该 PID	仍是几分钟前确认的实例；业务健康	用户、lstart、PPID、完整 args
kill 返回成功	发送请求被内核接受	进程已经退出；数据已安全落盘	等待后再次 ps，并按原特征 pgrep
kill -0 成功	PID 存在且当前用户有发送权限	身份正确、状态健康、终态正确	完整身份字段和业务验证
Z 状态	子进程已退出，父进程尚未回收	继续 kill -9 可以清除	PPID、父进程健康和僵尸增长趋势
D 状态	正处于不可中断等待	更强信号会立即生效	wchan、/proc/PID/io、共同挂载或设备
load average 高	可运行或不可中断等待任务较多	CPU 使用率一定高	多轮 top、R/D 数量、线程和短命进程
VSZ 很大	虚拟地址空间较大	已占用同等物理内存	RSS、系统可用内存、趋势和映射
原 PID 消失	原进程表项已不存在	业务实例不会再次出现	原匹配条件、PPID/祖先、启动时间
nice 值改变	普通调度倾向已变化	获得固定 CPU 份额；根因被修复	同窗口 CPU/负载采样和业务影响

最终 Cheatsheet

```
# 身份与关系
ps -p PID -o pid,ppid,user,lstart,etimes,sid,pgid,tty,stat,ni,%cpu,%mem,rss,vsz,args
pstree -aps PID

# 精确选择并预览
pgrep -a -u USER -f 'STABLE_PATH .*KEY_ARGUMENT'

# 趋势与线程
top -b -d 2 -n 5 -w 200
top -b -H -p PID -d 2 -n 5 -w 200

# /proc 下钻
cat /proc/PID/status
tr '\0' ' ' < /proc/PID/cmdline; echo
cat /proc/PID/io
cat /proc/PID/wchan

# 作业控制
jobs -l
bg %N
fg %N

# 安全终止
kill -TERM PID
sleep 2
ps -p PID -o pid,ppid,user,lstart,stat,args
# 重新确认身份和影响后，最后才考虑: kill -KILL PID

# 普通调度倾向
nice -n 10 COMMAND
renice --priority 15 --pid PID
```

向下一章交接

本章把一个运行实例识别为“可能受外部管理的进程树”，但不负责定义持久服务终态。进入下一章《systemd Unit、服务与依赖关系》后，需要把这里的证据继续映射到 unit：由哪个 unit 创建进程、主 PID 与工作进程是什么关系、为什么进程会被重新拉起，以及如何通过 unit 状态而不是追逐 PID 完成控制与验收。

进入下一章时继续回答：

1. 哪个 unit 是这棵进程树的实际管理者？
2. MainPID、工作进程与 Restart= 策略怎样对应本章看到的 PID 变化？
3. “进程存在”“service active”和“业务功能正常”分别需要什么证据？
4. 停止、启动、重启与开机启用为什么是不同的状态维度？

本章不会提前展开 unit 文件、依赖顺序、启用状态和 drop-in。这里保留的交接证据是：完整身份、进程树、启动时间、原业务匹配条件，以及“原 PID 消失后是否出现替代实例”的观察结果。